ARTIFICIAL INTELLIGENCE LABORATORY MANUAL

Search Algorithms & Expert Systems

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Table of Contents

EXPERIMENT 1

Breadth First Search (for 8 puzzle problem)

AIM

To implement Breadth First Search (BFS) to solve the 8-Puzzle problem by finding the shortest sequence of moves from the initial state to the goal state.

ALGORITHM

1. Start.
2. Initialize the queue with the initial state.
3. Mark the initial state as visited.
4. While the queue is not empty:
   a. Remove the front state from the queue.
   b. If it is the goal state, stop.
   c. Generate all possible next states by moving the blank tile.
   d. Add unvisited states to the queue and mark them visited.
5. Stop.

SOURCE CODE

```
from collections import deque

# Initial and Goal states
initial_state = (
    (1, 2, 3),
    (0, 4, 6),
    (7, 5, 8)
)

goal_state = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

# Possible moves: Up, Down, Left, Right
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def print_state(state):
    for row in state:
        print(row)
```

```
    print()

def bfs_8_puzzle():
    queue = deque()
    visited = set()

    # queue stores (current_state, path, depth)
    queue.append((initial_state, [], 0))
    visited.add(initial_state)

    while queue:
        current, path, depth = queue.popleft()
        path = path + [current]

        # Goal test
        if current == goal_state:
            print("Goal found at Depth:", depth)
            print("Solution Path:\n")

            for step in path:
                print_state(step)
            return

        x, y = find_blank(current)

        # Generate children (next level)
        for dx, dy in moves:
            nx, ny = x + dx, y + dy

            if 0 <= nx < 3 and 0 <= ny < 3:
                new_state = [list(row) for row in current]

                new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

                new_state = tuple(tuple(row) for row in new_state)

                if new_state not in visited:
                    visited.add(new_state)
                    queue.append((new_state, path, depth + 1))

# Run BFS
bfs_8_puzzle()
```

OUTPUT

```
Goal found at Depth: 3

Solution Path:

(1, 2, 3)
(0, 4, 6)
(7, 5, 8)

(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

RESULT

Thus the Program for Breadth First Search (BFS) to solve the 8-Puzzle problem was executed and output is verified successfully.

EXPERIMENT 2

Depth First Search (for Water Jug problem)

AIM

To write a program to implement water jug problem using Depth First Search.

ALGORITHM

1. Start with both jugs empty (0, 0).
2. Check whether the target amount is reached in any jug.
3. If the current state is already visited, return.
4. Mark the current state as visited.
5. Generate all possible next states by:
   i. Filling a jug
   ii. Emptying a jug
   iii. Pouring water from one jug to another
6. Apply DFS recursively on each valid unvisited state.
7. Continue until the target amount is obtained or all states are explored.
8. Stop.

SOURCE CODE

```
from collections import defaultdict

# Dictionary to keep track of visited states
visited = defaultdict(lambda: False)

# Jug capacities
jug1 = 4
jug2 = 3

# Target amount
aim = 2

def waterJugSolver(amt1, amt2):

    # Goal condition
    if (amt1 == aim and amt2 == 0) or (amt2 == aim and amt1 == 0):
        print(amt1, amt2)
        return True

    # If this state has not been visited
    if not visited[(amt1, amt2)]:
        print(amt1, amt2)
        visited[(amt1, amt2)] = True
```

```
        return (
            waterJugSolver(0, amt2) or
            waterJugSolver(amt1, 0) or
            waterJugSolver(jug1, amt2) or
            waterJugSolver(amt1, jug2) or

            waterJugSolver(
                amt1 + min(amt2, jug1 - amt1),
                amt2 - min(amt2, jug1 - amt1)
            ) or

            waterJugSolver(
                amt1 - min(amt1, jug2 - amt2),
                amt2 + min(amt1, jug2 - amt2)
            )
        )

    else:
        return False

# Driver code
print("Steps:")
waterJugSolver(0, 0)
```

OUTPUT

```
Steps:
0 0
4 0
1 3
1 0
0 1
4 1
2 3
2 0
```

RESULT

Thus Water Jug Program has been executed successfully.

EXPERIMENT 3

A* algorithm

AIM

To find the shortest path from Arad to Bucharest using the A* search algorithm.

ALGORITHM

1. Start from the initial node Arad.
2. Place the start node in the OPEN list.
3. Compute the evaluation function:

   $$f(n) = g(n) + h(n)$$

4. Select the node with the lowest f(n) from the OPEN list.
5. Expand the selected node and generate its neighboring nodes.
6. Update the cost values for each neighbor.
7. Move the expanded node to the CLOSED list.
8. Repeat until Bucharest is reached.
9. Stop.

SOURCE CODE

```
import heapq

# Romania Map Graph
graph = {
    'Arad': [('Zerind', 75), ('Timisoara', 118), ('Sibiu', 140)],
    'Zerind': [('Arad', 75), ('Oradea', 71)],
    'Oradea': [('Zerind', 71), ('Sibiu', 151)],
    'Timisoara': [('Arad', 118), ('Lugoj', 111)],
    'Lugoj': [('Timisoara', 111), ('Mehadia', 70)],
    'Mehadia': [('Lugoj', 70), ('Drobeta', 75)],
    'Drobeta': [('Mehadia', 75), ('Craiova', 120)],
    'Craiova': [('Drobeta', 120), ('Rimnicu Vilcea', 146), ('Pitesti', 138)],
    'Sibiu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Rimnicu Vilcea': [('Sibiu', 80), ('Craiova', 146), ('Pitesti', 97)],
    'Fagaras': [('Sibiu', 99), ('Bucharest', 211)],
    'Pitesti': [('Rimnicu Vilcea', 97), ('Craiova', 138), ('Bucharest', 101)],
    'Bucharest': []
}

# Heuristic values
heuristic = {
    'Arad': 366,
    'Zerind': 374,
    'Oradea': 380,
    'Timisoara': 329,
```

```
    'Lugoj': 244,
    'Mehadia': 241,
    'Drobeta': 242,
    'Craiova': 160,
    'Sibiu': 253,
    'Rimnicu Vilcea': 193,
    'Fagaras': 178,
    'Pitesti': 98,
    'Bucharest': 0
}

def a_star(start, goal):

    pq = []

    heapq.heappush(pq, (heuristic[start], 0, start, [start]))

    visited = set()

    while pq:

        f, g, current, path = heapq.heappop(pq)

        if current == goal:
            print("Optimal Path:")
            print(" → ".join(path))
            print("Total Cost:", g)
            return

        if current in visited:
            continue

        visited.add(current)

        for neighbor, cost in graph[current]:

            if neighbor not in visited:

                new_g = g + cost
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    pq,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

# Run A*
a_star('Arad', 'Bucharest')
```

OUTPUT

```
Optimal Path:
Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest
Total Cost: 418
```

RESULT

Thus, the shortest path from Arad to Bucharest was executed and output is verified successfully using the A* search algorithm.

EXPERIMENT 4

AO* (AO Star) algorithm

AIM

To implement the AO* (AO Star) algorithm for finding the optimal solution graph in an AND–OR graph using heuristic information.

ALGORITHM

1. Start with the initial node.
2. Assign heuristic values to all nodes.
3. Place the start node in the OPEN list.
4. Select the most promising node based on minimum estimated cost.
5. If the selected node is an OR node, choose the child with the minimum cost.
6. If the selected node is an AND node, choose all its children and sum their costs.
7. Update the cost of the parent node by backpropagation.
8. Mark the node as solved if all its successors are solved.
9. Repeat the above steps until the start node is solved.
10. Stop.

SOURCE CODE

```
# AO* Algorithm Implementation for the given AND-OR graph

# Graph representation
# Each node has groups of successors
# Each group = OR / AND choice

graph = {
    'A': [[('B', 5)], [('C', 3), ('D', 4)]],
    'B': [[('E', 10)], [('F', 11)]],
    'C': [[('G', 3)], [('H', 0), ('I', 0)]],
    'D': [[('J', 1)]],
    'E': [], 'F': [], 'G': [], 'H': [], 'I': [], 'J': []
}

# Heuristic / leaf costs
heuristic = {
    'A': 0,
    'B': 5,
    'C': 3,
    'D': 4,
    'E': 10,
    'F': 11,
    'G': 3,
    'H': 0,
```

```
    'I': 0,
    'J': 1
}

solved = {}

def AOStar(node):

    # If node already solved
    if node in solved:
        return solved[node]

    # If leaf node
    if not graph[node]:
        solved[node] = (heuristic[node], node)
        return solved[node]

    min_cost = float('inf')
    best_path = node

    # Evaluate all successor groups
    for group in graph[node]:

        cost = 0
        path = node

        # AND nodes → sum costs
        for child, edge_cost in group:

            child_cost, child_path = AOStar(child)

            cost += edge_cost + child_cost

            path += " -> " + child_path

        # Select minimum cost option
        if cost < min_cost:
            min_cost = cost
            best_path = path

    # Backpropagation
    solved[node] = (min_cost, best_path)

    return solved[node]

# Run AO*
cost, path = AOStar('A')

print("Optimal Solution Graph:", path)
print("Minimum Cost:", cost)
```

OUTPUT

```
Optimal Solution Graph: A -> C -> H -> I -> D -> J
Minimum Cost: 9
```

RESULT

Thus, the AO* algorithm efficiently found the optimal solution is executed and output is verified successfully.

EXPERIMENT 5

Single Player Game (Using Heuristic Function)

AIM

To implement a single player game for solving the 8-puzzle problem using a heuristic function, in order to reach the goal state efficiently from the given initial state.

ALGORITHM

1. Define the initial state and goal state.
2. Initialize a priority queue.
3. Compute the heuristic value for the initial state.
4. Insert the initial state into the priority queue.
5. Repeat until the queue is empty:
   a. Remove the state with minimum heuristic value
   b. If the state is the goal, stop
   c. Generate all valid successor states
   d. Compute heuristic values for successors
   e. Insert unvisited successors into the queue
6. End when the goal state is reached.

SOURCE CODE

```
from queue import PriorityQueue

# Initial State
START = [
    [1, 2, 3],
    [5, 6, 0],
    [7, 8, 4]
]

# Goal State
GOAL = [
    [1, 2, 3],
    [5, 8, 6],
    [0, 7, 4]
]

# Heuristic Function: Misplaced Tiles
def heuristic(state):

    h = 0

    for i in range(3):
        for j in range(3):
```

```
            if state[i][j] != 0 and state[i][j] != GOAL[i][j]:
                h += 1

    return h

# Find blank position
def find_blank(state):

    for i in range(3):
        for j in range(3):

            if state[i][j] == 0:
                return i, j

# Generate next states
def generate_states(state):

    x, y = find_blank(state)

    moves = [(-1,0), (1,0), (0,-1), (0,1)]

    children = []

    for dx, dy in moves:

        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:

            new_state = [row[:] for row in state]

            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

            children.append(new_state)

    return children

# Best First Search using heuristic
def single_player_game():

    pq = PriorityQueue()

    pq.put((heuristic(START), START))

    visited = []

    while not pq.empty():

        h, current = pq.get()

        print("State with h(n) =", h)

        for row in current:
            print(row)

        print("-----")
```

```
        if current == GOAL:
            print("Goal State Reached")
            return

        visited.append(current)

        for child in generate_states(current):

            if child not in visited:
                pq.put((heuristic(child), child))

# Run
single_player_game()
```

OUTPUT

```
State with h(n) = 2
[1, 2, 3]
[5, 6, 0]
[7, 8, 4]
-----

State with h(n) = 1
[1, 2, 3]
[5, 0, 6]
[7, 8, 4]
-----

State with h(n) = 0
[1, 2, 3]
[5, 8, 6]
[0, 7, 4]
-----

Goal State Reached
```

RESULT

A single player game for solving the 8-puzzle problem using a heuristic function was executed and output is verified successfully.

EXPERIMENT 6

Two Player Game (Using Heuristic Function)

AIM

To implement a two player game for the Tic-Tac-Toe problem using a heuristic function, where two players take turns and the computer selects optimal moves based on heuristic evaluation.

ALGORITHM

1. Initialize an empty 3×3 Tic-Tac-Toe board.
2. Assign Player O as the human player and Player X as the computer player.
3. Display the current board.
4. Allow the human player to make a valid move.
5. Check for a win or draw condition.
6. Generate all possible moves for the computer player.
7. Apply the heuristic function to evaluate each possible move.
8. Select the move with the best heuristic value.
9. Update the board and display the result.
10. Repeat the steps until a player wins or the game ends in a draw.

SOURCE CODE

```
# Tic-Tac-Toe: Two Player Game using Heuristic Function

board = [' ' for _ in range(9)]

# Display board
def show_board():

    print()

    print(board[0], '|', board[1], '|', board[2])
    print('--+---+--')
    print(board[3], '|', board[4], '|', board[5])
    print('--+---+--')
    print(board[6], '|', board[7], '|', board[8])

    print()

# Check win
def win(player):

    wins = [
        (0,1,2),(3,4,5),(6,7,8),
        (0,3,6),(1,4,7),(2,5,8),
        (0,4,8),(2,4,6)
```

```
    ]

    for a,b,c in wins:

        if board[a] == board[b] == board[c] == player:
            return True

    return False

# Heuristic function
def heuristic():

    if win('X'):
        return 10

    if win('O'):
        return -10

    return 0

# Check draw
def draw():
    return ' ' not in board

# Minimax algorithm
def minimax(is_max):

    score = heuristic()

    if score != 0:
        return score

    if draw():
        return 0

    if is_max:

        best = -1000

        for i in range(9):

            if board[i] == ' ':

                board[i] = 'X'

                best = max(best, minimax(False))

                board[i] = ' '

        return best

    else:

        best = 1000

        for i in range(9):
```

```
            if board[i] == ' ':

                board[i] = 'O'

                best = min(best, minimax(True))

                board[i] = ' '

        return best

# Computer move
def computer_move():

    best_val = -1000
    best_pos = -1

    for i in range(9):

        if board[i] == ' ':

            board[i] = 'X'

            move_val = minimax(False)

            board[i] = ' '

            if move_val > best_val:

                best_val = move_val
                best_pos = i

    board[best_pos] = 'X'

# Game loop
while True:

    show_board()

    # Human move
    pos = int(input("Enter position (0-8): "))

    if board[pos] != ' ':
        print("Invalid move")
        continue

    board[pos] = 'O'

    if win('O'):
        show_board()
        print("Player O wins!")
        break

    if draw():
        show_board()
        print("Game Draw!")
```

```
        break

    # Computer move
    computer_move()

    if win('X'):
        show_board()
        print("Player X (Computer) wins!")
        break
```

OUTPUT

```
Enter position (0-8): 0

O |   |
--+---+--
  | X |
--+---+--
  |   |

Enter position (0-8): 1

O | O | X
--+---+--
  | X |
--+---+--
  |   |

Player X (Computer) wins!
```

RESULT

Thus a two player game for the Tic-Tac-Toe problem using a heuristic function was executed and output is verified successfully.

EXPERIMENT 10

Expert system for Medical diagnosis

AIM

To develop a rule-based Expert System for Medical Diagnosis using Artificial Intelligence techniques that analyzes patient symptoms and predicts possible diseases with confidence levels using forward chaining inference mechanism.

ALGORITHM

1. Start
2. Display system title
3. Define list of symptoms
4. Accept user input for each symptom
5. Store symptoms in dictionary
6. Load disease rules (Knowledge Base)
7. For each disease rule:
   a. Count matched symptoms
   b. Calculate confidence percentage
   c. If confidence ≥ 60%, Display disease and confidence
8. If no disease matched, Display “Consult Doctor”
9. Stop

SOURCE CODE

```
print("======================================")
print("     ADVANCED MEDICAL EXPERT SYSTEM   ")
print("======================================")

# ------------------------------
# Step 1: Define Symptoms
# ------------------------------

symptom_list = [
    "fever", "high_fever", "cough", "dry_cough",
    "sore_throat", "runny_nose", "body_pain",
    "headache", "vomiting", "nausea",
    "diarrhea", "abdominal_pain", "chest_pain",
    "breathlessness", "fatigue", "weight_loss",
    "frequent_urination", "excessive_thirst",
    "sweating", "chills", "skin_rash",
    "joint_pain", "loss_of_smell",
    "loss_of_taste", "yellow_eyes",
    "weakness", "back_pain", "dizziness",
    "blurred_vision", "loss_of_appetite"
]
```

```
# ------------------------------
# Step 2: Take User Input
# ------------------------------

symptoms = {}

print("\nPlease answer with YES or NO:\n")

for symptom in symptom_list:

    answer = input(f"Do you have {symptom.replace('_',' ')}? (yes/no): ").lower()

    symptoms[symptom] = True if answer == "yes" else False

# ------------------------------
# Step 3: Knowledge Base (Rules)
# ------------------------------

diseases = {

    "Flu": ["fever", "cough", "body_pain", "fatigue"],

    "Malaria": ["high_fever", "chills", "sweating", "headache", "vomiting"],

    "COVID-19": ["fever", "dry_cough", "breathlessness", "loss_of_smell", "loss_of_taste"],

    "Heart Disease": ["chest_pain", "breathlessness", "sweating", "dizziness"],

    "Diabetes": ["frequent_urination", "excessive_thirst", "weight_loss", "fatigue",
"blurred_vision"],

    "Typhoid": ["fever", "abdominal_pain", "weakness", "loss_of_appetite"],

    "Dengue": ["high_fever", "joint_pain", "skin_rash", "headache", "vomiting"],

    "Jaundice": ["yellow_eyes", "fatigue", "abdominal_pain", "nausea"],

    "Food Poisoning": ["vomiting", "diarrhea", "abdominal_pain", "nausea"],

    "Common Cold": ["runny_nose", "sore_throat", "cough"],

    "Migraine": ["headache", "nausea", "dizziness"],

    "Kidney Infection": ["back_pain", "fever", "vomiting"]
}

# ------------------------------
# Step 4: Diagnosis Logic
# ------------------------------

print("\n======================================")
print("           DIAGNOSIS RESULT           ")
print("======================================")

found = False
```

```
for disease, rule_symptoms in diseases.items():

    match_count = 0

    for symptom in rule_symptoms:

        if symptoms[symptom]:
            match_count += 1

    confidence = (match_count / len(rule_symptoms)) * 100

    if confidence >= 60:

        found = True

        print(f"\nPossible Disease: {disease}")
        print(f"Confidence Level: {confidence:.2f}%")

# ------------------------------
# Step 5: If No Disease Found
# ------------------------------

if not found:

    print("\nNo strong disease match found.")
    print("Please consult a doctor for proper medical examination.")

print("\nThank you for using the Medical Expert System.")
```

OUTPUT

```
======================================
    ADVANCED MEDICAL EXPERT SYSTEM
======================================

Please answer with YES or NO:

Do you have fever? (yes/no): yes
Do you have cough? (yes/no): yes
Do you have body pain? (yes/no): yes
Do you have fatigue? (yes/no): yes

======================================
          DIAGNOSIS RESULT
======================================

Possible Disease: Flu
Confidence Level: 100.00%

Thank you for using the Medical Expert System.
```

RESULT

> The Medical Expert System was successfully executed and output is verified.